Data Structures & Algorithms Assignment # 2 (CLO-2.1)

Objective

Students will explore:

- To understand the implementation of singly, doubly, and circular linked lists.
- To apply Stack ADT and its operations in a real-world scenario.
- To develop problem-solving skills using pointers, dynamic memory, and linked data structures.
- To design and implement a menu-driven C++ application using data structures.
- To improve logical thinking, coding skills, and structured problem-solving.

Instructions

Make sure to follow these instructions carefully to avoid losing marks.

Topic: Your assignment is based on the topic:

"Linked Lists (Singly, Doubly, and Circular), Stack ADT, Pointers, and Dynamic Memory Allocation"

Format

- Students must write the explanations, algorithms, and diagrams by hand on paper.
- The C++ implementation must be completed in a `.cpp` file.
- Students must scan their handwritten work and combine it into a single PDF file.
- Submit both the handwritten PDF and C++ source code through the Moellim portal.

Problem Statement

Scenario: Hospital Patient Management System

A hospital wants to develop a Patient Management System to manage patient registration, treatment history, appointments, and emergency cases.

You are required to design and implement a menu-driven C++ program using singly linked lists, doubly linked lists, circular linked lists, and stacks.

Assignment Structure & Guidelines

Q1. Singly Linked List – Patient Registration (2 Marks)

Design a patient registration system using a singly linked list.

Each patient record should contain:

- Patient ID
- Patient Name
- Age
- Disease

Explain and implement the following operations:

- Add a new patient.
- Search for a patient by ID.
- Remove a patient by ID.
- Display all registered patients.

Q2. Doubly Linked List – Patient History (2 Marks)

The hospital wants to maintain a history of treated patients.

- Explain the structure of a doubly linked list.
- Add treated patients to the history.
- Display patient history in forward order.
- Display patient history in reverse order.

Draw a diagram showing how the `previous` and `next` pointers connect the nodes.

Q3. Circular Linked List – Appointment Management (2 Marks)

The hospital wants to manage patient appointments in a circular sequence.

- Add patients to the appointment list.
- Display all appointments.
- Move to the next patient's appointment.
- Explain how the last node connects back to the first node.

Draw a circular linked list diagram using at least four patient records.

Q4. Stack ADT – Emergency Management (2 Marks)

The hospital uses a stack to manage emergency patient entries.

Explain and implement the following operations:

- Push: Add an emergency patient.
- Pop: Remove the most recently added emergency patient.
- Peek: Display the top emergency patient.
- Display: Show all emergency patients.

Explain the LIFO (Last In, First Out) principle with an example from the hospital system.

Q5. Menu-Driven Program and Demonstration (2 Marks)

Develop a menu-driven C++ program that integrates all the above data structures.

Your program should allow hospital staff to select and perform operations for:

1. Patient registration.
2. Patient history.
3. Appointment management.
4. Emergency management.

Demonstrate the program using at least five patient records and provide sample output.

```
==========================================
     HOSPITAL PATIENT MANAGEMENT SYSTEM
==========================================
1. Patient Registration
2. Patient History
3. Appointment Management
4. Emergency Management
5. Exit

Enter your choice:
```

Submission

- Submit the assignment in PDF format and `.cpp` file via the Moellim portal.
- File name format: `SAP_ID_Data Structures_Assignment #2`
- This is an individual assignment. Plagiarism or copied code will result in zero marks.
- Deadline: To be announced by the instructor.
- Late submissions will incur a 5% deduction per day.

Output: Rubrics (Total = 10 Marks)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (2)	Poor (1)
Linked Lists	Correct implementation of all three types and operations.	Minor implementation errors.	Basic implementation with some errors.	Incomplete implementation.	Not attempted.
Stack ADT	Correct stack operations and LIFO explanation.	Minor errors in operations.	Basic stack implementation.	Incomplete operations.	Not attempted.
Program Logic	Complete menu-driven program with correct output.	Mostly correct program.	Basic working program.	Major logical errors.	Not attempted.
Presentation	Clear diagrams, explanations, and organized code.	Minor presentation errors.	Acceptable presentation.	Poor structure.	Unreadable.

Marks Distribution

Criteria	Marks
Singly Linked List – Q1	2
Doubly Linked List – Q2	2
Circular Linked List – Q3	2
Stack ADT – Q4	2
Menu-Driven Program – Q5	2
Total	10